

The Syntax of Sudoku (Grids) – Triplets

Draft! comments welcome

Alex Pfaff
nonintersective@gmail.com

December 28, 2025

Contents

0	Prologue	2
0.1	Empirical vs. np-rical (… and NP-rical)	2
0.2	Labeling Permutations: abc-Encoding and abc-Reduction	4
0.3	Preview of prospective attractions (cheat sheet)	7
1	Introduction	8
1.1	What is a triplet?	8
1.2	Permutations, Transformations, Properties, Triplet Counts	11
2	$n = 3$: Minimal Triplet Grids	14
3	$n = 4$: Impossible Grids	22
4	$n = 10$: Existential Uncertainty	23
4.1	$n = 10$: The General Problem	23
4.2	Attempt at a formal proof	24
5	n in General – Insanity! and more Empirical Data	25
5.1	Distinct Collections of n Triplets	28
5.2	Empty Intersections given n Triplets	30
5.3	Compatible Trios given n Triplets	31
5.4	Pairwise Union Identities given n Triplets	32
5.5	Balanced Occurrences	33
6	Triplet Puzzles	36

0 Prologue

This is a study into certain geometric properties of Sudoku grids involving permutations, construction and enumeration of possible grids. Now, the notion *empirical/experimental mathematics* probably does not ring a bell nor sound appealing to some readers, but I think it is an apt description of things to come. For the most part, we will not offer rigid proofs, but motivate computations that lead to certain numbers, even though, occasionally, we may resort brute force strategies (see Sect. 0.1).

0.1 Empirical vs. np-rical (... and NP-rical)

Every (empirical) science runs on data — data obtained from experiments, observations, measurements, field work, and the like. Sudokology is a special case in this respect, because its domain of study is finite and its objects are 100% subject to deterministic rules: there is no fuzziness, no measurement imprecision. The objects of interest are, of course, **grids, classes/families of grids**, as well as sub-grid components such as **boxes, rows, columns ...** and as we will see: **triplets**. Data is ideally obtained by following rigid mathematical/algorithmic procedures, e.g. *construct a grid (family of grids ...)* by (repeatedly) executing the following steps: 1. ... 2.

Sudoku, however, being an NP-complete problem poses a number of, well, problems, and a number of well-behaved algorithms and neat solutions to certain problems are still taxpayers and law-abiding citizens of that undiscovered country from whose bounds no traveler returns. Sudoku grids themselves are not usually observed in nature, and it's fair to say that launching questionnaires asking people about their favourite grids is rather pointless, so we need to get our data elsewhere.

empirical data. One approach is to create a (large) sample of randomly generated *valid* Sudoku grids, to analyse and classify them, and to hope that some generalisations emerge. This approach is closely related to the notion of *empirical probability* in the field of stochastics. A textbook example is flipping a coin: since a (fair) coin has two sides — head and tail — both with equal chances of showing after a toss, the hardwired probability for either outcome is 50%. In a real-life experiment, however, we may find that after n tosses, heads appears in only 42% of the cases; the expectation is that, as n increases, the observed probabilities will converge towards the hardwired ones.

For Sudoku, matters are more complicated. In many cases, we do not even know the hardwired probabilities in advance — the combinatorics of Sudoku is rather more sophisticated than that of coin tossing — but the hope is that sufficiently large samples will nevertheless yield empirically meaningful patterns and hints.

np-rical data. A somewhat intermediate approach: we generate datasets by (fully deterministic) algorithms that may, however, rely on semi-brute-force strategies and deliberately overgenerate, followed by the filtering of redundant data (an elegant algorithm would ideally not produce undesirable data in the first place). The relevant tools

stem from the Python library `PseudoQ`,^{1, 2} which relies heavily on `numpy`; the latter has the conventional alias **np** – hence the designation *np-rical*. One such application is the generation of some set of grids with subsequent **abc-reduction**, which potentially filters out redundant grids (see next section).

¹Accessible here:<https://github.com/A-Lex-McLee/PseudoQ-2.1>

²Actually, `PseudoQ` also delivers our “empirical” data; but those are generated with randomized strategies and therefore yield different outputs across runs. We still think it is a useful and insightful distinction.

0.2 Labeling Permutations: abc-Encoding and abc-Reduction

At what level are Sudoku grids identical or distinct? We may have a collection of grids compare them cell by cell (top-down, left-right, row-wise), and find that they represent distinct sequences:

(5, 7, 8, 4, 2, 9, 3, 1, 6, 9, 6, 2, 3, 5, 1, 7, 4, 8, 3, 4, 1, 7, 6)

(2, 7, 8, 4, 5, 9, 6, 1, 3, 9, 3, 5, 6, 2, 1, 7, 4, 8, 6, 4, 1, 7, 3)

.

Yet, we may examine an unnecessarily large collection to no avail, and worse, we may miss relevant generalizations. Consider the following four grids:

(1)

1 2 3 4 5 6 7 8 9
4 5 6 7 8 9 2 3 1
7 8 9 2 3 1 4 5 6
2 3 1 5 6 4 8 9 7
5 6 4 9 7 8 3 1 2
8 9 7 3 1 2 5 6 4
3 1 2 6 4 5 9 7 8
6 4 5 8 9 7 1 2 3
9 7 8 1 2 3 6 4 5

(2)

3 2 1 5 6 4 8 7 9
5 6 4 8 7 9 2 1 3
8 7 9 2 1 3 5 6 4
2 1 3 6 4 5 7 9 8
6 4 5 9 8 7 1 3 2
7 9 8 1 3 2 6 4 5
1 3 2 4 5 6 9 8 7
4 5 6 7 9 8 3 2 1
9 8 7 3 2 1 4 5 6

(3)

9 8 7 6 5 4 3 2 1
6 5 4 3 2 1 8 7 9
3 2 1 8 7 9 6 5 4
8 7 9 5 4 6 2 1 3
5 4 6 1 3 2 7 9 8
2 1 3 7 9 8 5 4 6
7 9 8 4 6 5 1 3 2
4 6 5 2 1 3 9 8 7
1 3 2 9 8 7 4 6 5

(4)

7 4 1 5 2 8 9 6 3
5 2 8 9 6 3 4 1 7
9 6 3 4 1 7 5 2 8
4 1 7 2 8 5 6 3 9
2 8 5 3 9 6 1 7 4
6 3 9 1 7 4 2 8 5
1 7 4 8 5 2 3 9 6
8 5 2 6 3 9 7 4 1
3 9 6 7 4 1 8 5 2

Anticipating some terminology, we observe that the first three of those grids involve the *triplets* $\langle 1, 2, 3 \rangle$, $\langle 4, 5, 6 \rangle$, $\langle 7, 8, 9 \rangle$, whereas the fourth comprises completely different triplets, $\langle 1, 4, 7 \rangle$, $\langle 2, 5, 8 \rangle$, $\langle 3, 6, 9 \rangle$. Cellwise comparison reveals that these are four distinct grids.

However, all these grids (and many more) are identical at a deeper (= structural or distributional) level; they all translate to the same **abc-grid**:

	A	B	C		D	E	F		G	H	I	
	D	E	F		G	H	I		B	C	A	
	G	H	I		B	C	A		D	E	F	
	B	C	A		E	F	D		H	I	G	
	E	F	D		I	G	H		C	A	B	
	H	I	G		C	A	B		E	F	D	
	C	A	B		F	D	E		I	G	H	
	F	D	E		H	I	G		A	B	C	
	I	G	H		A	B	C		F	D	E	

Abc-grids are a special kind of encoding, where an *abc-grid* encodes the structural distribution independently of symbol choice. Encoding first extracts a key by mapping the top row of a given grid to the fixed alphabetic sequence *ABCDEFGHI*:

encoder key:

from	9	8	7	6	5	4	3	2	1	top row of a given grid
to	A	B	C	D	E	F	G	H	I	fixed sequence

and consequently replaces the digits in the entire grid by letters according to this key. The resulting abc-grid encodes a deep identity in terms of structure/distribution.

Recoding, conversely, creates a key by mapping the alphabetic sequence to some ordering of the digits 1 – 9:

recoder key:

from	A	B	C	D	E	F	G	H	I	fixed sequence
to	1	2	3	4	5	6	7	8	9	some ordering of digits 1 – 9

and then substitutes the letters accordingly. Since there are $9! = 362,880$ such orderings ($\Leftrightarrow$ valid Sudoku rows), each abc-grid can be recoded in 362,880 ways.

More specifically, this means **one abc-grid translates to 362,880 distinct digit grids**; these digit grids are still distributionally identical, and essentially *are* the same grid; the distinctness is superficial. So when examining structural properties of such a collection of grids, it is actually sufficient to look at one representative. In principle, this representative could be any one out of 362,880 digit grids, but in order to unambiguously make the intention clear we will usually pick the corresponding abc-grid as representative. An abc-grid can be regarded as the ID of the grid family. Other than the neat geometric insights – if we have large collections of grids that may contain grids from distinct families, this notion is quite powerful because it allows us to only consider distinct abc-grids, which **potentially reduces the search space / runtime by a factor of 362,880**.

The operation of recoding is an instance of *label permutation* that preserves structure. In the context of `PseudoQ`, the set of $9!$ recodings / label permutations is referred to as the *horizontal (permutation) series*. This contrasts with the *vertical (permutation) series*. Vertical permutation is the set of grid-level permutations (swapping rows, columns, boxrows,³ boxcolumns); it is invasive and changes structure, and therefore produces $3!^4 \times 3!^4 = 6^8 = 1296 \times 1296 = 1,679,616$ genuinely distinct abc-grids. We will return to horizontal and vertical permutation in Sect. 1.2.

³A terminological notes: instead of *stack*, *band*, we will use the more transparent terms *boxcolumn*, *boxrow* when referring to the vertical or horizontal sequence of three boxes. Following Sudoku conventions, the numbering starts top/left; so e.g. boxcolumn 2 $\Leftrightarrow$ boxes 2, 5, 8 $\Leftrightarrow$ columns 4, 5, 6.

0.3 Preview of prospective attractions

(cheat sheet)

⇒ **Set of triplets:**

$$\begin{aligned}\mathbb{T}^* &= \{ \langle 1, 2, 3 \rangle, \langle 1, 2, 4 \rangle \dots \langle 5, 6, 7 \rangle, \langle 5, 6, 8 \rangle \dots \langle 6, 8, 9 \rangle, \langle 7, 8, 9 \rangle \} \\ |\mathbb{T}^*| &= \binom{9}{3} = 84\end{aligned}$$

⇒ **Set of combinations of n triplets:**

$$\begin{aligned}\mathbb{T}^n &= \dots \\ |\mathbb{T}^n| &= \binom{84}{n} \\ \mathbb{T}^3 &= \{ \{ \langle 1, 2, 3 \rangle, \langle 1, 3, 7 \rangle, \langle 4, 6, 8 \rangle \}, \{ \langle 2, 3, 4 \rangle, \langle 5, 7, 8 \rangle, \langle 6, 7, 8 \rangle \} \dots \} \\ |\mathbb{T}^3| &= \binom{84}{3} = 95,284\end{aligned}$$

⇒ **Set of combinations of n triplets that lead to valid grids:**

$$\begin{aligned}\mathbb{T}_g^n &= ? \\ |\mathbb{T}_g^n| &= ? \\ \mathbb{T}_g^3 &= \{ \{ \langle 1, 2, 3 \rangle, \langle 4, 5, 6 \rangle, \langle 7, 8, 9 \rangle \}, \{ \langle 1, 4, 7 \rangle, \langle 2, 5, 8 \rangle, \langle 3, 6, 9 \rangle \} \dots \}\end{aligned}$$

⇒ **Set of valid grids with n triplets:**

$$\begin{aligned}\mathbb{G}_T^n &= ? \\ |\mathbb{G}_T^n| &= ? \\ |\mathbb{G}_T^3| &= (\text{see Sect. 2})\end{aligned}$$

⇒ **Finally** (or actually: hopefully):

$$\sum_{n=3}^{27} |\mathbb{G}_T^n| = |\mathbb{G}| = 6,670,903,752,021,072,936,960 \approx 6.7 \times 10^{21}$$

1 Introduction

1.1 What is a triplet?

Assume we are given a (valid) Sudoku grid g (*valid* meaning: the digits 1–9 occur exactly once in each row, column, and box):

5 2 1 9 7 6 8 3 4
8 3 6 1 4 2 5 9 7
4 9 7 5 8 3 1 6 2
2 1 5 7 3 9 6 4 8
3 6 8 4 2 5 9 7 1
9 7 4 8 6 1 3 2 5
1 5 2 3 9 7 4 8 6
6 8 3 2 5 4 7 1 9
7 4 9 6 1 8 2 5 3

Proceeding row-wise, we may decompose g into 27 contiguous segments of length 3 (the width of a box). For example, the first few segments are

$(5, 2, 1), (9, 7, 6), (8, 3, 4), (8, 3, 6), (1, 4, 2), (5, 9, 7), \dots, (6, 1, 8), (2, 5, 3).$

Segments are *concrete occurrences*: they are ordered triples tied to specific positions in a specific grid. By contrast, we define a **triplet** as an abstraction over segments, whose identity is determined solely by **membership**, not by order or location.

A triplet could hence be represented either as a set or as a canonically ordered tuple, but to distinguish triplets from concrete segments, we will use angle brackets: $\langle \dots \rangle$.

Segment	Set	Ordered tuple	Triplet
$(5, 2, 1)$	$\{2, 5, 1\}$	$(1, 2, 5)$	$\langle 1, 2, 5 \rangle$

This definition immediately allows for *cross-segment identity*: distinct segments may instantiate the same triplet. For instance, consider box-column 1. It contains $3 \times 3 = 9$ segments, but these reduce to only three distinct triplets:

$(5, 2, 1), (2, 1, 5), (1, 5, 2) \Leftrightarrow \langle 1, 2, 5 \rangle,$

$(8, 3, 6), (3, 6, 8), (6, 8, 3) \Leftrightarrow \langle 3, 6, 8 \rangle,$

$(4, 9, 7), (9, 7, 4), (7, 4, 9) \Leftrightarrow \langle 4, 7, 9 \rangle.$

Thus, while every valid grid contains exactly 27 segments, the number of *distinct* triplets may vary. The grid g shown above, for example, contains 15 unique triplets:

$\langle 6, 7, 9 \rangle, \langle 2, 4, 5 \rangle, \langle 5, 7, 9 \rangle, \langle 1, 2, 6 \rangle, \langle 4, 6, 8 \rangle,$
 $\langle 2, 3, 5 \rangle, \langle 3, 4, 8 \rangle, \langle 3, 7, 9 \rangle, \langle 3, 5, 8 \rangle, \langle 3, 6, 8 \rangle,$
 $\langle 1, 2, 5 \rangle, \langle 4, 7, 9 \rangle, \langle 1, 7, 9 \rangle, \langle 1, 6, 8 \rangle, \langle 1, 2, 4 \rangle$

Since a classical 9×9 Sudoku grid contains 27 segments, the **maximum** possible number of distinct triplets is 27. At the other extreme, a **minimum of 3 triplets** is required: their union must cover all 9 digits in order for the grid to be valid at all.

It is plausible that not all values of n (the number of unique triplets) occur with equal frequency. To explore this, a sample of **111,230,620** randomly generated valid Sudoku grids was partitioned into classes indexed by n , and the cardinality $|cls_n|$ of each class was recorded.

triplet count class n	attestations of grids class size $ cls_n $	order of magnitude
25	22,883,329	$\sim 10^7$
24	18,517,142	
23	15,075,995	
26	13,247,241	
20	8,647,525	$\sim 10^6$
21	7,785,875	
22	7,150,343	
19	6,238,520	
27	5,054,871	
18	2,635,381	
17	1,725,354	
15	939,704	$\sim 10^5$
14	638,052	
16	346,101	
13	196,148	
12	67,571	$\sim 10^4$
11	37,500	
9	29,114	
8	9,347	$\sim 10^3$
7	4,508	
5	621	$\sim 10^2$
3	206	
6	172	
10	0	
4	0	

Table 1: Number of unique triplets per grid (n) versus observed grid counts in a large random sample. Classes are ranked by decreasing frequency.

Given the size of the full Sudoku universe ($\approx 6.7 \times 10^{21}$ valid grids), this sample is necessarily tiny, and strict representativity cannot be claimed. Nevertheless, as a heuristic first probe, we rely on `random` and `numpy.random` to generate sufficiently diverse instances for broad structural tendencies to emerge.

At first glance, there is a coarse positive correlation between n and $|cls_n|$: grids with many triplets are more frequent than those with few. This is unsurprising, since the number of available triplets grows combinatorially: there are $\binom{9}{3} = 84$ possible

triplets, $\binom{84}{n}$ combinations of n triplets, and the number of ways to assemble larger collections grows rapidly as well.

However, the ranking is far from monotonic. For example, $|cls_{25}|$ is the most frequent class, while $|cls_{27}|$ appears only in ninth position; $|cls_3|$ greatly exceeds $|cls_6|$; and $|cls_{23}|$ is significantly larger than $|cls_{26}|$.

One might suspect that these irregularities are artefacts of sampling, and that a truly representative sample (or the full universe) would enforce a strict ordering

$$|cls_{27}| \gg |cls_{26}| \gg |cls_{25}| \gg \dots \gg |cls_3|.$$

Yet this need not be the case. Increasing n also interacts with the constraints imposed by Sudoku rules, plausibly in a non-linear manner. Certain values of n may restrict grid construction, while others may inadvertently facilitate it, leading to local increases or decreases in $|cls_n|$. On this interpretation, Table 1 may already reflect genuine structural effects.

Several numerical properties of n suggest themselves as potentially relevant:

- (i) $n = 3k$ or $n = 9k$ (multiples of 3 or 9),
- (ii) $n \equiv m \pmod{3}$ or $n \equiv m \pmod{9}$ (residue classes),
- (iii) $n = q^2$ (square numbers),
- (iv) $n \in / \notin \mathbb{P}$ (primality).

Primality is one of the “usual suspects” in mathematics, but in the present context its relevance is far from obvious (though we will not dismiss it outright just yet). The square shape of the classical 9×9 grid might suggest a special role for square numbers, yet this intuition already falters: $n = 4$ is impossible, whereas $n = 25$ not only exists but is the most frequent class observed.

By contrast, properties (i) and (ii) are compelling candidates. The numbers 3 and 9 are hard-wired into the very architecture of Sudoku, and it would be surprising if divisibility and congruence with respect to these numbers played no role. We will return to this point in due course.

The grouping of classes by powers of ten in Table 1 is merely an optical convenience. In what follows, our attention will focus on four particularly revealing meta-classes:

- (i) $n = 3$ $\rightarrow$ directly enumerable and constructible,
- (ii) $n = 4$ $\rightarrow$ impossible,
- (iii) $n = 10$ $\rightarrow$ existence? — the great conundrum,
- (iv) all remaining cases $\rightarrow$ an insane challenge.

1.2 Permutations, Transformations, Properties, Triplet Counts

As already mentioned, grid permutation produces 1,679,616 structurally distinct abc-grids ($\rightarrow$ the vertical series). It is interesting to note that *rotation* does not add new grids. That is, after applying all four rotations to all those 1,679,616 grids we still only have 1,679,616 distinct abc-grids. Obviously, rotations emerge as combinations of various (row/column/boxrow/boxcolumn) swappings, and thus the vertical series already contains all possible permutations. Strictly speaking, we could now filter out the rotations and reduce the vertical series to $1,679,616/4 = 419,904$ unique core grids, but for the moment, we will retain abc-distinctness as bottomline and consider grid orientation as a primitive. On this view, rotation will produce structurally different grids. Differently from rotation, *diaflexion* – reflection across the main diagonal (= matrix transposition) – leads to genuinely new grids. Thus, in conjunction, grid permutation + diaflexion produce $2 \times 1,679,616 = 3,359,232$ distinct abc-grids.

Apart from structural distinctness, Sudoku grids may have specific (geometric) properties, for instance:

- (i) grid satisfies **anti knight's move constraint**: two cells a chess knight's move apart (on a bounded grid) may not contain the same digit; **modular anti knight's move constraint**: as the former, but the grid is construed as a toroidal wrap-around structure, thus the knight can see across grid borders.
- (ii) (main / anti / both) **diagonal(s)** constitute(s) a separate Sudoku dimension, i.e. contain(s) 9 distinct digits, or contain(s) three times the same triplet ...
- (iii) the digits in the **Phistomefel ring** sum to a prime number, or (don't) contain certain digits ...
- (iv) ... (plus many more)

Now the question is: which property is preserved under which permutations/transformations? Horizontal permutation (relabeling) is structure preserving and thus preserves most properties – except cases where digit identity, i.e. its value, is relevant, e.g. property (iii) above, Killer Sudoku and other cases where specific cell values must sum to a given value. Vertical permutation is invasive and potentially destroys structural properties, for instance properties (i)-(iii).

Both horizontal and vertical permutation, however, preserve the triplet count. Horizontal permutation are bound to change triplet identities, e.g. $\langle 1, 2, 5 \rangle \rightarrow \langle 2, 4, 8 \rangle$, and vertical permutation the triplet positions, but if a given grid contains n triplets, it will still contain n triplets after horizontal/vertical permutation.

On the other hand, rotation by 90° and diaflexion change the triplet count. The reason is that we defined triplets as abstractions over row-wise segments, and applying either transformation will transform the original row-wise segments into column-wise segments (abstraction over which is not on the menu today), while we now abstract over the original column-wise segments, which usually leads to a different triplet count:⁴

⁴We may return to the question whether there are grids have the same triplet count under rotation.

original:	rotated by 90°
-----	-----
2 3 1 5 4 6 9 7 8	4 8 3 6 7 1 5 9 2
9 8 7 1 2 3 4 5 6	6 7 1 5 9 2 4 8 3
5 4 6 7 8 9 3 1 2	5 9 2 4 8 3 6 7 1
-----	-----
1 2 3 6 5 4 8 9 7	9 2 4 8 3 6 7 1 5
7 9 8 3 1 2 5 6 4	7 3 6 9 1 5 8 2 4
6 5 4 8 9 7 1 2 3	8 1 5 7 2 4 9 3 6
-----	-----
3 1 2 4 6 5 7 8 9	2 6 7 1 5 8 3 4 9
8 7 9 2 3 1 6 4 5	3 4 8 2 6 9 1 5 7
4 6 5 9 7 8 2 3 1	1 5 9 3 4 7 2 6 8
-----	-----
unique triplets:	unique triplets:
<1, 2, 3>	<1, 5, 7>,
<4, 5, 6>	<1, 5, 8>,
<7, 8, 9>	<1, 5, 9>,
	<1, 6, 7>,
	<2, 4, 7>,
	<2, 4, 8>,
	<2, 4, 9>,
	<2, 5, 9>,
	<2, 6, 7>,
	<2, 6, 8>,
	<2, 6, 9>,
	<3, 4, 7>,
	<3, 4, 8>,
	<3, 4, 9>,
	<3, 6, 7>,
	<3, 6, 8>,
	<3, 6, 9>

With that in mind, we may now precisify our first task as follows: for a given triplet count n , can we construct a core set of abc-distinct grids that are not related by grid permutation? If such a core set exists, it can subsequently be expanded by applying vertical permutations, potentially increasing its size by a factor of 6^8 .⁵ This staged approach is computationally far more tractable than attempting to construct all grids of triplet count n in one monolithic enumeration.

The deeper methodological question, however, is one of completeness. We must ask whether this construction risks missing genuine grids of triplet count n , i.e. valid grids that cannot be reduced to any element of the core set via abc-reduction and grid permutation. Put differently: are there additional transformations—beyond horizontal and vertical permutation—that preserve triplet count and should therefore be factored into our equivalence relation? At present, horizontal and vertical permutations

⁵In principle, horizontal permutation would yield an additional factor of $9!$; however, since horizontal permutation corresponds merely to digit relabeling, it generates only notational variants and no new structural cases. We therefore usually suppress this step.

exhaust the class of transformations that provably preserve triplet count as defined here. Rotations and diaflections do not, in general, preserve this invariant, because they interchange row-wise and column-wise segment structure. Any further admissible transformations would therefore have to respect the row-wise segmentation underlying the triplet abstraction itself. Whether such transformations exist in a nontrivial form, and whether they generate genuinely new equivalence classes, remains an open question—and one that can ultimately only be settled empirically by examining the stability of triplet counts under progressively enlarged transformation sets.

2 $n = 3$: Minimal Triplet Grids

We begin with the simplest nontrivial case. Assume we are given exactly three distinct triplets $T_1, T_2, T_3 \in \mathbb{T}^3$, with the property that together they exhaust the digit set:

$$T_1 \cup T_2 \cup T_3 = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}, \quad T_i \cap T_j = \emptyset \text{ for } i \neq j.$$

For concreteness, and without loss of generality, let

$$T_1 = \langle 1, 2, 3 \rangle,$$

$$T_2 = \langle 4, 5, 6 \rangle,$$

$$T_3 = \langle 7, 8, 9 \rangle.$$

Triplets as row-wise building blocks. By design, any concatenation of the three triplets yields a valid Sudoku row, regardless of order or internal permutation. For instance:

$$[(1, 2, 3), (4, 5, 6), (7, 8, 9)] \quad [T_1, T_2, T_3]$$

$$[(4, 5, 6), (7, 8, 9), (1, 2, 3)] \quad [T_2, T_3, T_1]$$

$$[(9, 7, 8), (2, 3, 1), (6, 5, 4)] \quad [T_3, T_1, T_2]$$

Vertical stacking and box structure. The same observation applies vertically. Stacking (permutations of) the three triplets produces a valid 3×3 box:

$$\begin{bmatrix} (1, 2, 3) \\ (4, 5, 6) \\ (7, 8, 9) \end{bmatrix}, \quad \begin{bmatrix} (4, 5, 6) \\ (7, 8, 9) \\ (1, 2, 3) \end{bmatrix}, \quad \begin{bmatrix} (9, 7, 8) \\ (2, 3, 1) \\ (6, 5, 4) \end{bmatrix}$$

$$\begin{bmatrix} T_1 \\ T_2 \\ T_3 \end{bmatrix}, \quad \begin{bmatrix} T_2 \\ T_3 \\ T_1 \end{bmatrix}, \quad \begin{bmatrix} T_3 \\ T_1 \\ T_2 \end{bmatrix}$$

Triplet grid templates. The procedure is deterministic: Once one triplet is given, there are only two possible continuations within the respective row/box, and as long as triplets are not repeated, validity is guaranteed at the (local) row/box level. Since a *boxcolumn* is essentially three boxes stacked, it follows that each boxcolumn contains each triplet three times. Combining these observations straightforwardly leads to the construction of a *grid template*, in which each row and each box contains all three

triplets exactly once:

T_1	T_2	T_3
T_2	T_3	T_1
T_3	T_1	T_2
T_1	T_2	T_3
T_2	T_3	T_1
T_3	T_1	T_2
T_1	T_2	T_3
T_2	T_3	T_1
T_3	T_1	T_2

This template already satisfies all Sudoku constraints at the triplet level. What remains is to choose *how* each triplet is internally permuted. One possible instantiation of this template is shown below:

1 2 3 4 5 6 7 8 9
4 5 6 7 8 9 3 1 2
7 8 9 1 2 3 5 6 4
2 3 1 5 6 4 8 9 7
5 6 4 8 9 7 2 3 1
8 9 7 2 3 1 6 4 5
3 1 2 6 4 5 9 7 8
6 4 5 9 7 8 1 2 3
9 7 8 3 1 2 4 5 6

This grid uses only the three triplets T_1, T_2, T_3 according to the template above, repeated throughout in different permutations.

Column constraints and permutation compatibility. At this point, an important subtlety enters. While rows and boxes are guaranteed to be valid by construction, column validity depends on how the triplets are permuted; recall we have 3×3 triplets per boxcolumn. Each triplet has $3! = 6$ internal permutations ($\rightarrow$ segments). These permutations fall naturally into two compatibility classes. For $T_1 = \langle 1, 2, 3 \rangle$, for example:

(Class A) $(1, 2, 3), (2, 3, 1), (3, 1, 2)$ vs. $(1, 3, 2), (2, 1, 3), (3, 2, 1)$ (Class B)

Permutations within the same class can be stacked vertically without causing column repetitions; mixing classes generally leads to conflicts. In the above grid, all permutations of T_1 belong to class A; substituting any of those with a permutation from class B leads to a violation of the column constrain in the respective boxcolumn:

$(1, 2, 3)$ $(1, 2, 3)$ $(1, 3, 2)$ $(1, 3, 2)$

(2, 3, 1)	(2, 1, 3)	(2, 3, 1)	(2, 1, 3)
(3, 1, 2)			(3, 2, 1)

Thus, for each triplet and boxcolumn, one compatibility class must be chosen. Once chosen, the three segments, in turn, can be ordered (vertically!) in $3! = 6$ ways, which is illustrated for class A below:

(1, 2, 3)	(1, 2, 3)	(2, 3, 1)	(2, 3, 1)	(3, 1, 2)	(3, 1, 2)
(2, 3, 1)	(3, 1, 2)	(1, 2, 3)	(3, 1, 2)	(1, 2, 3)	(2, 3, 1)
(3, 1, 2)	(2, 3, 1)	(3, 1, 2)	(1, 2, 3)	(2, 3, 1)	(1, 2, 3)

Structural versus labeling permutations. The two compatibility classes differ only by a relabeling of digits (e.g. swapping 2 and 3):

• (1, 2, 3)	[swap values: $2 \mapsto 3; 3 \mapsto 2$]	$\Rightarrow$	(1, 3, 2)
(2, 3, 1)	[swap values: $2 \mapsto 3; 3 \mapsto 2$]	$\Rightarrow$	(3, 2, 1)
(3, 1, 2)	[swap values: $2 \mapsto 3; 3 \mapsto 2$]	$\Rightarrow$	(2, 1, 3)

This motivates a useful distinction:

- *Structural permutations*, which affect column compatibility;
- *Labeling permutations*, which merely rename symbols.

Labeling permutation entails that class B can be reduced to class A (or vice versa) without loosing deep structural distinctions. So effectively, we need to consider only one compatibility set per triplet – per boxcolumn, not at grid level.

At the mere triplet level, this distinction, may be of questionable benefit, but at the grid level, it reveals its full power. This is where abc-grids (re-)enter the stage; horizontal permutation / recoding is effectively just the set of labeling permutations on one abc-grid. In Sect. 0.2, it was shown that abc-encoding effectively reduces the search space by a factor of 362,880 since we can ignore the superficial label permutations.

Constructing and enumerating grids – the core set: So the economic question we need to ask: **How many distinct abc-grids with $n = 3$ triplets can we construct?** (and how?). Different digit assignments (recodings) can be handled after that, separately. This task is still of some complexity, and we will therefore proceed step-wise: Let us first (re-)state our assumptions and then construct a core set:

- (i) $T_1 = \langle a, b, c \rangle; T_2 = \langle d, e, f \rangle; T_3 = \langle g, h, i \rangle;$

abc-grids are (conceptually) immutable in the sense that they cannot be themselves relabeled ($\text{str} \mapsto \text{str}$); swapping, say, 'A's and 'B's will produce an alphabetized grid that is valid Sudoku-wise, but it is no longer an abc grid in the strict sense; the defining top row is no longer preserved. We will use lowercase letters to indicate that we are dealing with “aspiring” abc-grids at this stage.

- (ii) top row of boxcolumn 1: (a, b, c)
top row of boxcolumn 2: (d, e, f)
top row of boxcolumn 3: (g, h, i)
- (iii) As shown, the two compatibility classes can be reduced to one (via labeling permutations); at the construction stage we will make use of only one compatibility class per triplet – grid-wide!
- (iv) Since the top row (segment) is fixed in every boxcolumn, every boxcolumn contains one triplet with only **two admissible distributions**:

boxcol 1: (a, b, c)	boxcol 2: (d, e, f)	boxcol 3: (g, h, i)
(b, c, a)	(e, f, d)	(h, i, g)
(c, a, b)	(f, d, e)	(i, g, h)
boxcol 1: (a, b, c)	boxcol 2: (d, e, f)	boxcol 3: (g, h, i)
(c, a, b)	(f, d, e)	(i, g, h)
(b, c, a)	(e, f, d)	(h, i, g)

- (v) The remaining two triplets per boxcolumn, respectively, each have the full range of $3! = 6$ **distributions**.
- (vi) On this count, we actually have $2 \times 6 \times 6 = 72$ possible valid boxcolumns.
- (vii) The prospective abc-grid already determines the top row as $[T_1, T_2, T_3]$ and thus reduces the range of possible continuations, box-wise. We stipulate the same ordering – ... $T_3, T_1, T_2, T_3, T_1, T_2$... – vertically, effectively determining a template for each boxcolumn, which yields the grid template already illustrated above:

$T_1,$	$T_2,$	$T_3,$
$T_2,$	$T_3,$	$T_1,$
$T_3,$	$T_1,$	$T_2,$
$T_1,$	$T_2,$	$T_3,$
$T_2,$	$T_3,$	$T_1,$
$T_3,$	$T_1,$	$T_2,$
$T_1,$	$T_2,$	$T_3,$
$T_2,$	$T_3,$	$T_1,$
$T_3,$	$T_1,$	$T_2]$

This way we ensure that every triplet occurs exactly once per row and per box (and three times per boxcolumn).

- (viii) Since (these) boxcolumns are independent of each other, valid grids are now ensured by the Cartesian product $boxcol_1 \times boxcol_2 \times boxcol_3$, which amounts to $72 \times 72 \times 72 = 72^3 = 373,248$ valid abc-grids.⁶

⁶Each of these grids, in turn, can be recoded ($\rightarrow$ label permutations) in $9!$ ways, thus at this stage, we can construct $362,880 \times 373,248 = 135,444,234,240$ valid Sudoku grids with $n = 3$ triplets.

The grids so generated are already valid abc-grids and cannot be further abc-reduced; moreover they are genuinely distinct. So our core set does indeed contain 373,248 grids. Still it is valid to ask: Did we miss anything? Notably, didn't we lose possible grids by only considering two distributions per boxcolumn for the triplet that occurs in the respective top row? Let's check: Omitting points (ii/iv) has two consequences:

- Our set includes non-canonical alphabetized grids (i.e. non-abc-grids):

[(b, a, c), (d, e, f), (g, h, i)]
 [(a, b, c), (f, e, d), (g, h, i)]
 [(a, b, c), (d, e, f), (i, g, h)] ... etc.

- We raise the count of possible valid boxcolumns to $6 \times 6 \times 6 = 216$, and, consequently, generate $216 \times 216 \times 216 = 10,077,696$ grids.

The collection generated does indeed contain 10077696 distinct (valid) grids! However, if we re-abc-encode all grids in this collection and then count unique (distinct) grids, we once again arrive at **373,248** grids. This shows that the additionally generated grids are mere notational variants (label permutations / recodings) of the ones we generate by the procedure as described above. We were right to restrict the construction the way we did, i.e. fixing the top row and thus exempting the triplets involved from permutation.⁷

Moreover, it impressively illustrates the power of abc-encoding, notably, its power of reduction and simplification! Incidentally, this approach falls under the heading *np*-*practical*: overgenerate and then reduce/filter (in the given case, we can actually prevent it by rigidly following steps (i)–(viii)). We will see more of that in the following.

It seems we have exhausted the range of low-level permutations, let's now attend to grid-level permutations: swapping of columns, boxcolumns, rows, boxrows, each of which has three primitives,⁸ which produces $3!^3 \times 3! \times 3!^3 \times 3! = 6^8 = 1,679,616$ distinct (abc-)grids from a given grid ($\rightarrow$ the vertical permutation series), see Sects. 0.2, 1.2.

Column permutations. Given a valid grid, columns within a boxcolumn can be permuted in $3! = 6$ ways, each permutation yielding another valid grid. Assume we have a given abc-grid from which we generate the column permutations within boxcolumn 1 (i.e. cols 1, 2, 3); as expected, we get 6 distinct grids. Obviously, column permutation disrupts the canonical abc ordering; if we now re-abc-encode these 6 grids, we likewise obtain 6 distinct abc-grids (ignore the numbers on top of those grids for now):

(1) column permutations of some abc-grid

⁷Under the hood, it could have been any other row; fixing the top row in abc-grids is a mere convention.

⁸ $\Rightarrow$ three boxrows, three boxcolumns, three rows within one boxrow / three columns within one boxcolumn (permuting across boxes is illegal since it usually leads to one or more Sudoku violations).

A B C D E F G H I	A C B D E F G H I	B A C D E F G H I
D E F G H I A B C	D F E G H I A B C	E D F G H I A B C
G H I A B C D E F	G I H A B C D E F	H G I A B C D E F
B C A E F D H I G	B A C E F D H I G	C B A E F D H I G
E F D H I G B C A	E D F H I G B C A	F E D H I G B C A
H I G B C A E F D	H G I B C A E F D	I H G B C A E F D
C A B F D E I G H	C B A F D E I G H	A C B F D E I G H
F D E I G H C A B	F E D I G H C A B	D F E I G H C A B
I G H C A B F D E	I H G C A B F D E	G I H C A B F D E
B C A D E F G H I	C A B D E F G H I	C B A D E F G H I
E F D G H I A B C	F D E G H I A B C	F E D G H I A B C
H I G A B C D E F	I G H A B C D E F	I H G A B C D E F
C A B E F D H I G	A B C E F D H I G	A C B E F D H I G
F D E H I G B C A	D E F H I G B C A	D F E H I G B C A
I G H B C A E F D	G H I B C A E F D	G I H B C A E F D
A B C F D E I G H	B C A F D E I G H	B A C F D E I G H
D E F I G H C A B	E F D I G H C A B	E D F I G H C A B
G H I C A B F D E	H I G C A B F D E	H G I C A B F D E

(2) column permutations re-abc-encoded

1 1 1		
A B C D E F G H I	A B C D E F G H I	A B C D E F G H I
D E F G H I A B C	E F D G H I C A B	F D E G H I B C A
G H I A B C D E F	H I G C A B D E F	I G H B C A D E F
B C A E F D H I G	B C A E F D H I G	B C A E F D H I G
E F D H I G B C A	F D E H I G A B C	D E F H I G C A B
H I G B C A E F D	I G H A B C E F D	G H I C A B E F D
C A B F D E I G H	C A B F D E I G H	C A B F D E I G H
F D E I G H C A B	D E F I G H B C A	E F D I G H A B C
I G H C A B F D E	G H I B C A F D E	H I G A B C F D E
1 2 2		
A B C D E F G H I	A B C D E F G H I	A B C D E F G H I
D F E G H I A C B	E D F G H I B A C	F E D G H I C B A
G I H A C B D E F	H G I B A C D E F	I H G C B A D E F
C A B E F D H I G	C A B E F D H I G	C A B E F D H I G
E D F H I G C B A	F E D H I G A C B	D F E H I G B A C
H G I C B A E F D	I H G A C B E F D	G I H B A C E F D
B C A F D E I G H	B C A F D E I G H	B C A F D E I G H
F E D I G H B A C	D F E I G H C B A	E D F I G H A C B
I H G B A C F D E	G I H C B A F D E	H G I A C B F D E

If we apply these column permutations to the entire core set, we get $373,248 \times 6 = 2239488$ (valid) distinct grids, as expected. However, if we now re-abc-encode this collection of grids once again and count the unique grids, we only get $746,496 = 373248 \times 2$ unique grids; we observe a partial abc-reduction. Apparently, certain

abc-grids obtained from re-abc-encoding of certain column permutations – which in isolation produces new (unique) grids – are already members of the core set.⁹ We mentioned a similar effect in the context of rotation applied to the vertical series; different (permutation/transformation) paths can lead to the same grid.

Conjecture. Effectively, column permutation only doubles the number of distinct grids, and there is a likely reason for this factor of 2: Above it was shown that the set of permutations divides into two compatibility classes in terms of possible continuations. It was also shown that – at the level of boxcolumns – the two can effectively be reduced to one since they are notational variants of each other. At the grid-level, however, this is no longer true because, within the grid, the boxcolumns are independent of each other (by design). Therefore, permutations applied locally do not affect the global domain. Consider the following distribution of a triplet in three boxcolumns:

(1, 2, 3)	(1, 2, 3)	(1, 2, 3)
(2, 3, 1)	(3, 1, 2)	(2, 3, 1)
(3, 1, 2)	(2, 3, 1)	(3, 1, 2)

Applying some local permutation like swapping columns 2 and 3 only affects boxcolumn 1, which potentially leads to a structural deviance or innovation that cannot be “fixed” via relabeling:

(1, 3, 2)	(1, 2, 3)	(1, 2, 3)
(2, 1, 3)	(3, 1, 2)	(2, 3, 1)
(3, 2, 1)	(2, 3, 1)	(3, 1, 2)

The re-abc-encoded grids in 2 are divided into two groups: 1 1 1 → the same compatibility class in all boxcolumns (top); 1 2 2 → boxcolumn 1 has a different compatibility class (bottom).

In other words, relabeling now operates at the level of the entire grid, but here the two compatibility classes become a contrastive structural feature. While all permutations in compatibility class A are already in the core set – by design (steps (i)-(viii) above), the introduction of compatibility class B locally brings about a genuine innovation. Therefore, **only compatibility class survives abc-reduction**, which leaves us with the observed factor 2.

The insanity does not end here: once we accept that permutation within one boxcolumn + abc-reduction merely doubles the count of distinct grids, we should expect that the conjunction of all legal column permutations

$$[\text{perm}(c1, c2, c3) \times \text{perm}(c4, c5, c6) \times \text{perm}(c7, c8, c9)](\text{core set})$$

⁹or emerge from other permutations of other grids.

plus subsequent abc-reduction will lead to a count of $373,248 \times 2^3 = 2,985,984$ distinct grids. However, we only count **1,492,992** unique abc-grids, which corresponds to a factor of 4 (not 8). Let's take it more slowly: if we apply two sets of column permutations, in any order, we do get 1,492,992 distinct abc-grids, which corresponds to a factor of $2 \times 2 = 4$, as predicted at this stage. It is the respective third set of column permutations applied to this set that only produces notational variants, and thus effectively does not increase the count of distinct abc-grids.

preliminary counts:

```
check\_permCount()
tripletGrids.shape: (373248, 9, 9)
```

```
raw count boxrows:          (2239488, 9, 9)
unique count boxrows:       (373248, 81)      ==> x 1

raw count rows perms (0, 1, 2): (2239488, 9, 9)
unique count rows perms (0, 1, 2): (2239488, 81)      ==> x 6

raw count rows perms (3, 4, 5): (2239488, 9, 9)
unique count rows perms (3, 4, 5): (2239488, 81)      ==> x 6

raw count rows perms (6, 7, 8): (2239488, 9, 9)
unique count rows perms (6, 7, 8): (2239488, 81)      ==> x 6
-----

raw count boxcols:          (2239488, 9, 9)
unique count boxcols:       (746496, 81)      ==> x 2

raw count col perms (0, 1, 2): (2239488, 9, 9)
unique count col perms (0, 1, 2): (746496, 81)      ==> x 2

raw count col perms (3, 4, 5): (2239488, 9, 9)
unique count col perms (3, 4, 5): (746496, 81)      ==> x 2

raw count col perms (6, 7, 8): (2239488, 9, 9)
unique count col perms (6, 7, 8): (746496, 81)      ==> x 2
```

THE FINAL TWIST: FULL SET of GRID PERMUTATIONS!!!

currently, it seems that the final count of account abc-minTriplet-grids =

$373248 \times 1 \times 6 \times 6 \times 6 \times 2 \times 2 \times 2 \times 2 = \mathbf{1,289,945,088}$ –

is that the end of our adventure? Worry not! We will return to further calculations after the winter break! Be prepared for new twists and surprises; the mystery continues

3 $n = 4$: Impossible Grids

In Table 1, the entry for $n = 4$ is 0, i.e. no valid grid with exactly four distinct triplets is attested. At first glance, this absence might be dismissed as a sampling artefact: perhaps such grids exist, but are simply too rare to appear in the sample.

We will show, however, that this gap is *systemic*. In fact, a valid Sudoku grid consisting of exactly four distinct triplets cannot exist at all. The argument proceeds by contradiction.

Proof by contradiction.

Assume that there exists a valid Sudoku grid built from **four distinct triplets** T_1, T_2, T_3, T_4 . Without loss of generality, let us assume that three of them are

$$T_1 = \langle 1, 2, 3 \rangle,$$

$$T_2 = \langle 4, 5, 6 \rangle,$$

$$T_3 = \langle 7, 8, 9 \rangle.$$

These three triplets already form a complete partition of the digits 1–9. Consequently, any row composed of one permutation of each of T_1 , T_2 , and T_3 is automatically a valid Sudoku row, regardless of the internal ordering:

$$[(1, 2, 3), (4, 5, 6), (7, 8, 9)],$$

$$[(4, 5, 6), (7, 8, 9), (1, 2, 3)],$$

$$[(9, 7, 8), (2, 3, 1), (6, 5, 4)], \quad \text{etc.}$$

Now consider the fourth triplet T_4 . Since the grid is assumed to contain *four* distinct triplets, T_4 must occur at least once. Wherever it occurs — in a row, column, or box-row — it must be accompanied by two further triplets so that all nine digits appear exactly once.

If T_4 cannot be combined with any two of $\{T_1, T_2, T_3\}$ without violating the Sudoku constraints, then the construction already fails.¹⁰

Suppose, then, that T_4 *can* be completed to a valid Sudoku row. There are only three possibilities:

$$[T_4, T_2, T_3] \text{ is valid} \quad \Rightarrow \quad T_4 = T_1,$$

$$[T_4, T_1, T_2] \text{ is valid} \quad \Rightarrow \quad T_4 = T_3,$$

$$[T_4, T_1, T_3] \text{ is valid} \quad \Rightarrow \quad T_4 = T_2.$$

The reason is simple: any valid Sudoku row partitions the digits 1–9 into three disjoint triplets. Since $\{T_1, T_2, T_3\}$ already form such a partition, replacing one of them forces the remaining triplet to be *exactly* the missing one.

¹⁰For example, if $T_4 = \langle 1, 4, 7 \rangle$, then any attempted combination with two of T_1, T_2, T_3 necessarily produces at least one repeated digit.

Thus, in order for T_4 to appear in a valid Sudoku dimension, it must be *identical* (in terms of membership) to one of T_1, T_2 , or T_3 .

But this contradicts the original assumption that there are **four distinct triplets**. Once T_4 coincides with any of the others, the grid in fact contains only **three** distinct triplets.¹¹

We conclude that a valid Sudoku grid with exactly four distinct triplets cannot exist: either the grid violates Sudoku constraints, or it collapses into the already admissible case $n = 3$.

q.e.d.

4 $n = 10$: Existential Uncertainty

4.1 $n = 10$: The General Problem

Unlike the case $n = 4$, the absence of grids with $n = 10$ distinct triplets does not immediately admit a clean structural explanation. In Table 1, the class $|cls_{10}|$ is empty: among more than 10^8 randomly generated valid Sudoku grids, not a single instance with exactly ten distinct triplets was observed.

At first sight, this raises a familiar question. Is $n = 10$ *impossible*, in the same strict sense as $n = 4$? Or is it merely *extremely rare*, perhaps too rare to surface in a finite random sample of this size?

At present, no valid grid with $n = 10$ triplets is known to the author. This negative evidence is suggestive, but not decisive. Unlike the case $n = 4$, there is no immediate contradiction that forces a collapse to a smaller value of n , nor does an obvious “identity crisis” arise from attempting to introduce a tenth triplet.

From a purely numerical point of view, $n = 10$ does not stand out in any trivial way. It satisfies

$$10 \equiv 1 \pmod{3},$$

but so do 7, 13, 16, 19, $\dots$, all of which are well attested in the data. If congruence modulo 3 were the decisive factor, one would expect a whole family of missing classes rather than a single isolated gap.

Nor does $n = 10$ carry any immediately suggestive arithmetic property: it is neither prime nor square, neither minimal nor maximal, and it lies comfortably within the densely populated mid-range of observed values. In this respect, its absence is more puzzling than that of $n = 4$.

This suggests that, if $n = 10$ is indeed impossible, the obstruction must be more subtle — not a simple counting argument, but a geometric or combinatorial incompatibility between

- the way triplets must be reused across rows,
- the necessity of covering all 27 row-wise segments,
- and the constraint that only ten distinct triplet identities are available.

¹¹Recall that triplet identity is defined by membership alone. Relabelling does not create a new triplet.

One way to phrase the problem is the following. A grid with $n = 10$ triplets must distribute 27 row-wise segments among exactly 10 equivalence classes. By the pigeon-hole principle alone, some triplets must occur frequently, others only sparsely. This uneven distribution may interact poorly with Sudoku's rigid local constraints: repetition across rows is harmless, but repetition within rows is forbidden, and repetition patterns propagate through boxes in non-trivial ways.

In contrast to the case $n = 3$, where extreme regularity makes construction straightforward, and to the case $n = 4$, where the attempt fails immediately, $n = 10$ appears to inhabit an uncomfortable middle ground: too many triplets to force global rigidity, too few to allow local flexibility.

There is also a faint analogy with the $n = 4$ case worth noting, though it is only heuristic. Both values satisfy $n \equiv 1 \pmod{3}$, meaning that the number of triplets does not decompose naturally into full Sudoku rows (each row consuming exactly three triplets). For $n = 4$, this mismatch leads directly to impossibility. For $n = 10$, the mismatch is less severe, but it may still impose hidden constraints on how rows can be assembled without eventually forcing unintended identifications or violations elsewhere in the grid.

Whether this intuition can be sharpened into a proof remains open. At the very least, the persistent absence of $n = 10$ across a very large and diverse sample suggests that this value is not merely rare in a probabilistic sense, but structurally disfavoured.

In the next subsection, we therefore turn from observation to construction. Rather than asking whether grids with $n = 10$ exist, we ask how one might try to build such a grid — and where, precisely, such attempts begin to break down.

4.2 Attempt at a formal proof

⇒ this will be your homework for next week ;)

5 n in General – Insanity! and more Empirical Data

Let's take a detour and consider some practical issues. The class `Grid` offers a.o. the following functionalities to generate a (valid) Sudoku grid at random:¹²

```
>>> from PsQ_Grid import Grid
>>> g = Grid()
>>> g.generate_rndGrid(seed=42)
>>> g.showGrid()
```

```
-----
| 4 9 8 | 3 2 1 | 6 5 7 |
| 3 6 5 | 7 9 8 | 2 4 1 |
| 1 2 7 | 4 5 6 | 8 3 9 |
-----
| 2 4 6 | 8 1 9 | 3 7 5 |
| 5 7 1 | 6 3 2 | 4 9 8 |
| 9 8 3 | 5 4 7 | 1 6 2 |
-----
| 7 5 4 | 2 8 3 | 9 1 6 |
| 8 3 9 | 1 6 5 | 7 2 4 |
| 6 1 2 | 9 7 4 | 5 8 3 |
-----
```

In addition, the random generator can be constrained: (i) anti knight's move constraint; (ii) modular anti knight's move constraint; (iii) one or both diagonal must contain distinct digits; all constraints can be imposed simultaneously:

```
>>> g.setGenerator(anti_knight=True)
>>> g.setGenerator(anti_knight=True, modular_knight=True)
>>> g.setGenerator(diagonal_choice="anti") # or 'main' or 'both'

>>> g.setGenerator(anti_knight=True, modular_knight=True, diagonal_choice="both")
>>> g.generate_rndGrid(seed=42)
>>> g.showGrid()
```

```
-----
| 4 9 8 | 3 2 1 | 6 5 7 |
| 1 5 3 | 7 9 6 | 8 2 4 |
| 6 2 7 | 4 5 8 | 3 9 1 |
-----
| 8 6 2 | 1 4 5 | 7 3 9 |
| 5 3 1 | 9 6 7 | 2 4 8 |
| 7 4 9 | 8 3 2 | 1 6 5 |
-----
| 2 7 4 | 5 8 3 | 9 1 6 |
| 3 1 5 | 6 7 9 | 4 8 2 |
| 9 8 6 | 2 1 4 | 5 7 3 |
-----
```

¹²The code can be downloaded from <https://github.com/A-Lex-McLee/PseudoQ-2.1>; for instructions, see *Grid_class_manual.pdf* also found there.

The final setting imposes some severe restrictions on a possible output making the computation quite complex; nonetheless, in maximally 3-5 seconds the desired output is delivered (worst case scenario). So three cheers for the computational power of modern computers and the efficiency of `numpy`!

Now let's attempt to construct the following function that receives an input value n and outputs a valid Sudoku grid with n distinct triplets ($\rightarrow \text{grid} \in \mathbb{G}_T^n$):

```
def generate_randomTripletGrid(n: int) -> np.ndarray:
```

.... and we are stuck! Empirical tests have shown that you are more likely to win the lottery three times in series than this function to return a valid output. Concrete implementation specifics aside, this is not accidental, and there is a reason why the general random grid generator (even with some constraints) works so well, whereas the random triplet grid generator does not. Although both problems concern the same combinatorial object, their computational character differs fundamentally.

Problem (a): Unconstrained Sudoku generation. In problem (a), the algorithm searches for any grid satisfying the local Sudoku constraints: each row, column, and 3×3 box must contain the digits $1, \dots, 9$ exactly once. These constraints are local, homogeneous, and symmetric across the grid. Crucially, the algorithm is free to accept *any* configuration that satisfies them.

From a combinatorial perspective, this problem benefits from the enormous size of the solution space (on the order of 10^{21} grids, modulo symmetries and reductions). Randomized construction or backtracking with constraint propagation is highly effective: partial assignments typically admit many valid extensions, and dead ends are pruned early. The algorithm is therefore performing an existence search in a large, well-connected feasible space, which explains why valid grids can be generated reliably and quickly in practice.

Problem (b): Sudoku generation with prescribed triplet structure. Problem (b) introduces a qualitatively different constraint. The grid must realize *exactly* n distinct digit triplets, drawn from the set of all

$$\binom{9}{3} = 84$$

possible triplets. This requirement is global rather than local: it constrains the multiset of patterns used across all 27 Sudoku dimensions (rows, columns, and boxes), not merely their individual validity.

For a given n , the algorithm must implicitly solve several coupled combinatorial subproblems:

1. **Triplet selection:** choose a subset of n triplets from 84 possibilities, a space of size $\binom{84}{n}$.
2. **Multiplicity constraints:** assign frequencies to these triplets so that they cover exactly 27 dimensions while maintaining balanced digit usage.

3. **Compatibility constraints:** ensure that selected triplets can be combined in triples whose union is $\{1, \dots, 9\}$, corresponding to valid rows, columns, or boxes.
4. **Geometric realizability:** embed these abstract combinations consistently into the fixed 9×9 Sudoku architecture, in particular satisfying column constraints that couple otherwise independent row and box constructions.

Each layer introduces combinatorial growth, but the principal difficulty arises from their *non-linear interaction*. A triplet collection may satisfy balance and compatibility conditions in isolation, yet still be unrealizable as a concrete Sudoku grid due to positional conflicts or column inconsistencies. As n varies, the density of feasible configurations changes sharply, producing irregular feasibility regions and forbidden gaps.

Fundamental source of the complexity gap. The essential distinction is that problem (a) is a pure existence problem in a vast solution space, whereas problem (b) is a conditional existence problem constrained by a high-dimensional global invariant. In problem (a), randomness and backtracking exploit the abundance of solutions; in problem (b), they contend with scarcity. The latter problem effectively requires solving a constrained design problem over a compatibility hypergraph, followed by a constrained embedding into a rigid geometric structure.

This explains why algorithms that perform extremely well for unconstrained Sudoku generation may fail or stall entirely when an exact triplet count is imposed. The resulting combinatorial explosions are not artifacts of implementation, but intrinsic to the global structure of the constrained problem.

Divide and conquer. Let's split the problem into two components: 1. the actual algorithm that attempts to assemble (permutations of) given triplets in accordance with the Sudoku constraints, and eventually return some grid in $\mathbb{G}_T^n$; 2. a procedure that produces a collection of n triplets in $\mathbb{T}_g^n$. In the following, we will address that latter point.

It is easy to randomly sample n triplets, for instance via the following function:

```
>>> def generate_triplets(n: int) -> np.array:
    triplets = np.array(list(combinations(range(1, 10), 3)))
    n_idx = np.random.choice(triplets.shape[0], n,
                             replace=False)
    return triplets[n_idx]
```

However, this function samples from all possible triplets, and it is far from impossible that, e.g. for $n=3$, we get the output $[(1, 2, 3), (1, 2, 4), (2, 4, 6)]$. Obviously, it is a priori impossible to construct a valid grid from these triplets, several digits are missing, and any attempt to use a backtracking to generate a grid would be a waste of computational energy and time. This highlights the importance of ensuring that the output really is in $\mathbb{T}_g^n$ before passing it on to any algorithm. Now digit completeness

$$T_1 \cup T_2 \cdots \cup T_n = \{1, 2, 3, 4, 5, 6, 7, 8, 9\},$$

is a low-level criterion easily checked and can well be baked into a function like `generate_triplets`. But obviously, this is not enough; there are several more factors to consider. Actually, at present, I am not aware of a single bullet-proof criterion that unambiguously classifies a given triplet collection of size n as $\in / \notin \mathbb{T}_g^n$ – other than extracting the triplets from a given valid grid whose very existence entails that its unique triples are in $\mathbb{T}_g^n$.

The strategy is thus: apply a series of criteria and any triplet collection that survives those is a (likely!) candidate $\in \mathbb{T}_g^n$. In order to give (numeric) substance to those criteria, we will once more turn to empirical data drawn from tens of millions of (randomly generated) valid (abc-encoded!) grids and run diagnostics for relevant criteria that must be satisfied. The results, in turn, will give us some parametric settings for the respective testing methods. In this respect, the procedure is of practical relevance, and there may be gaps because the sample is not large enough. At the same, certain gaps may not be accidental and they can give us insight into the complex combinatorial relation between n triplets and valid grids, or else can be posed as combinatorial (research) questions in themselves.

5.1 Distinct Collections of n Triplets

$\Rightarrow |\mathbb{T}_g^n|$?

As already mentioned, we apply abc-reduction thus sorting out grids that are mere notational variants already sampled. In addition, we additionally reduce by sorting. Triplets themselves are ordered by convention, but the function `get_uniqueTriplets()` returns the n unique triplets of a given grid in some (random) order:

$n = 13$ triplets in random order	$n = 13$ triplets in ascending order
(3, 4, 6)	(1, 2, 3)
(5, 7, 9)	(1, 2, 6)
(1, 2, 6)	(1, 2, 8)
(3, 4, 9)	(1, 2, 9)
(1, 2, 3)	(3, 4, 5)
(1, 2, 9)	(3, 4, 6)
(5, 7, 8)	(3, 4, 8)
(5, 6, 7)	(3, 4, 9)
(3, 4, 8)	(4, 5, 6)
(3, 4, 5)	(5, 6, 7)
(4, 5, 6)	(5, 7, 8)
(7, 8, 9)	(5, 7, 9)
(1, 2, 8)	(7, 8, 9)

One triplet collection $\in \mathbb{T}_g^n$ gives rise to many(!) distinct abc-grids; so if we looked at every collection as is by output, we would risk overcounting. Sorting allows us to identify identical triplet collections more easily and filter out duplicates. With that in mind, the following counts were observed for $n \in \{3, \dots, 27\}$:

3 : 1
4 : 0
5 : 26
6 : 16
7 : 122
8 : 615
9 : 1,577
10 : 0
11 : 1,332
12 : 2,818
13 : 8,476
14 : 26,749
15 : 32,973
16 : 12,236
17 : 55,994
18 : 83,113
19 : 145,330
20 : 180,944
21 : 175,001
22 : 169,608
23 : 304,372
24 : 359,244
25 : 427,535
26 : 263,155
27 : 93,382

5.2 Empty Intersections given n Triplets

$\Rightarrow$ how many instances of $T_i \cap T_j = \emptyset$?

Obviously, if two triplets occur in the same row or box, they must not have any overlap, otherwise the row or column could never be validly completed. At the very least we need three pairwise mutually exclusive triplets to complete any row or box, and this is the exact count for $n = 3$. Below are the attested counts for $n \in \{3, \dots, 27\}$:

- 3 : {3}
- 4 : { }
- 5 : {6}
- 6 : {8, 9}
- 7 : {9, 10, 11}
- 8 : {9, 11, 12, 13, 14}
- 9 : {9, 11, 12, 13, 14, 15, 16, 17, 18}
- 10 : { }
- 11 : {23, 24}
- 12 : {21, 25, 26, 27, 28}
- 13 : {28, 29, 30, 31, 32}
- 14 : {24, 26, 28, 29, 30, 31, 32, 33, 34, 35, 36}
- 15 : {30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41}
- 16 : {39, 40, 41, 43, 44, 45, 46}
- 17 : {41, 43, 44, 45, 46, 47, 48, 49, 50, 51}
- 18 : {39, 42, 44, 45, 46, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57}
- 19 : {49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63}
- 20 : {49, 51, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68}
- 21 : {54, 55, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74}
- 22 : {66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79}
- 23 : {66, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86}
- 24 : {66, 72, 73, 74, 75, 76, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92}
- 25 : {74, 78, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99}
- 26 : {78, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 107}
- 27 : {93, 94, 96, 97, 98, 99, 100, 101, 102, 103, 105, 106, 108, 109, 111, 114}

Very obviously, the relation is not a function, because there is no unambiguous mapping; range of attested numbers, see also the following

5.3 Compatible Trios given n Triplets

$\Rightarrow$ how many instances of $T_i \cup T_j \cup T_k = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$?

3 : {1}

4 : { }

5 : {2}

6 : {2}

7 : {3}

8 : {3}

9 : {3, 4, 6}

10 : { }

11 : {7}

12 : {7, 8}

13 : {8, 9, 10}

14 : {8, 9, 12}

15 : {8, 9, 10, 11, 12}

16 : {11, 12, 13}

17 : {12, 13, 14}

18 : {12, 13, 14, 15, 16}

19 : {13, 14, 15, 16, 17}

20 : {13, 14, 15, 16, 17, 18}

21 : {13, 14, 15, 16, 17, 18, 19, 21}

22 : {16, 17, 18, 19, 20}

23 : {17, 18, 19, 20, 21, 22, 24}

24 : {17, 18, 19, 20, 21, 22, 23}

25 : {18, 19, 20, 21, 22, 23, 24}

26 : {18, 19, 20, 21, 22, 23, 25}

27 : {18, 19, 20, 21, 22, 23, 24, 25, 27, 28, 36}

5.4 Pairwise Union Identities given n Triplets

$\Rightarrow$ how many instances of $T_i \cup T_j = T_k \cup T_l$? for $T_i \neq T_j \neq T_k \neq T_l$
 $\Rightarrow \langle \mathbf{T}_a, T_i, T_j \rangle$ is a valid row $\Leftrightarrow \langle \mathbf{T}_a, T_k, T_l \rangle$ is a valid row

- 3 : {0}
- 4 : { }
- 5 : {1}
- 6 : {0}
- 7 : {1, 2}
- 8 : {1}
- 9 : {0, 3, 9}
- 10 : { }
- 11 : {9}
- 12 : {9, 10, 11, 12}
- 13 : {9, 10, 11, 14}
- 14 : {9, 10, 11, 12, 13, 14, 16}
- 15 : {9, 10, 11, 12, 13, 14, 15}
- 16 : {14, 15, 16, 17, 20}
- 17 : {15, 17, 18, 19, 20, 21}
- 18 : {15, 16, 17, 18, 19, 20, 21, 22, 27}
- 19 : {17, 18, 19, 20, 21, 22, 23, 27}
- 20 : {17, 18, 19, 20, 21, 22, 23, 24, 25, 27}
- 21 : {18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28}
- 22 : {22, 23, 24, 25, 26, 27, 28, 29, 30, 32, 34}
- 23 : {23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34}
- 24 : {24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 36, 39}
- 25 : {25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40}
- 26 : {26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 43}
- 27 : {27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 45}

5.5 Balanced Occurrences

For a fixed triplet count n , the digit count sums to $3n$ (across the triplet collection, not grid). For every grid in the sample, we record the (sorted) 9-tuple of digit frequencies per n : $(df_1, df_2, df_3, df_4, df_5, df_6, df_7, df_8, df_9)$ where df_n means: digit n occurs df_n times in the triplet collection:

```
>>> dfs = np.unique(triplet_collection, return_count=True)[1]
>>> dfs = sorted(tuple(dfs))
>>> ...

3: (1, 1, 1, 1, 1, 1, 1, 1, 1)
4: ()
5: (1, 1, 1, 2, 2, 2, 2, 2, 2)
6: (2, 2, 2, 2, 2, 2, 2, 2, 2)
7: (1, 1, 1, 3, 3, 3, 3, 3, 3)
   (2, 2, 2, 2, 2, 2, 3, 3, 3)
8: (2, 2, 2, 3, 3, 3, 3, 3, 3)
9: (3, 3, 3, 3, 3, 3, 3, 3, 3)
10: ()
11: (3, 3, 3, 4, 4, 4, 4, 4, 4)
12: (3, 3, 4, 4, 4, 4, 4, 5, 5)
   (3, 4, 4, 4, 4, 4, 4, 4, 5)
   (4, 4, 4, 4, 4, 4, 4, 4, 4)
13: (3, 3, 3, 5, 5, 5, 5, 5, 5)
   (3, 3, 4, 4, 5, 5, 5, 5, 5)
   (3, 4, 4, 4, 4, 5, 5, 5, 5)
   (4, 4, 4, 4, 4, 4, 5, 5, 5)
14: (4, 4, 4, 4, 5, 5, 5, 5, 6)
   (4, 4, 4, 5, 5, 5, 5, 5, 5)
15: (3, 3, 5, 5, 5, 6, 6, 6, 6)
   (4, 4, 4, 5, 5, 5, 6, 6, 6)
   (4, 4, 5, 5, 5, 5, 5, 6, 6)
   (4, 5, 5, 5, 5, 5, 5, 6, 6)
   (5, 5, 5, 5, 5, 5, 5, 5, 5)
16: (4, 4, 5, 5, 5, 6, 6, 6, 6)
   (4, 5, 5, 5, 5, 6, 6, 6, 6)
   (5, 5, 5, 5, 5, 5, 6, 6, 6)
```

17 : (3, 3, 5, 6, 6, 7, 7, 7, 7)
 (4, 4, 5, 6, 6, 6, 6, 7, 7)
 (4, 4, 6, 6, 6, 6, 6, 6, 7)
 (5, 5, 5, 5, 5, 5, 7, 7, 7)
 (5, 5, 5, 5, 5, 6, 6, 7, 7)
 (5, 5, 5, 5, 6, 6, 6, 6, 7)
 (5, 5, 5, 6, 6, 6, 6, 6, 6)

18 : (4, 4, 5, 6, 7, 7, 7, 7, 7)
 (4, 4, 6, 6, 6, 7, 7, 7, 7)
 (4, 5, 5, 6, 6, 7, 7, 7, 7)
 (4, 6, 6, 6, 6, 6, 6, 7, 7)
 (5, 5, 5, 5, 6, 6, 7, 7, 8)
 (5, 5, 5, 6, 6, 6, 7, 7, 7)
 (5, 5, 6, 6, 6, 6, 6, 7, 7)
 (5, 6, 6, 6, 6, 6, 6, 6, 7)
 (6, 6, 6, 6, 6, 6, 6, 6, 6)

19 : (4, 4, 6, 6, 7, 7, 7, 8, 8)
 (4, 4, 6, 7, 7, 7, 7, 7, 8)
 (5, 5, 5, 7, 7, 7, 7, 7, 7)
 (5, 5, 6, 6, 7, 7, 7, 7, 7)
 (5, 6, 6, 6, 6, 7, 7, 7, 7)
 (6, 6, 6, 6, 6, 6, 7, 7, 7)

20 : (5, 5, 6, 6, 7, 7, 8, 8, 8)
 (5, 5, 6, 7, 7, 7, 7, 8, 8)
 (5, 5, 7, 7, 7, 7, 7, 7, 8)
 (5, 6, 6, 7, 7, 7, 7, 7, 8)
 (5, 6, 7, 7, 7, 7, 7, 7, 7)
 (6, 6, 6, 6, 7, 7, 7, 7, 8)
 (6, 6, 6, 7, 7, 7, 7, 7, 7)

21 : (3, 3, 7, 7, 7, 9, 9, 9, 9)
 (5, 5, 7, 7, 7, 8, 8, 8, 8)
 (6, 6, 6, 6, 7, 7, 7, 9, 9)
 (6, 6, 6, 7, 7, 7, 8, 8, 8)
 (6, 6, 7, 7, 7, 7, 7, 7, 9)
 (6, 6, 7, 7, 7, 7, 7, 8, 8)
 (6, 7, 7, 7, 7, 7, 7, 7, 8)
 (7, 7, 7, 7, 7, 7, 7, 7, 7)

22 : (4, 4, 7, 7, 8, 9, 9, 9, 9)
 (4, 4, 7, 8, 8, 8, 9, 9, 9)
 (4, 7, 7, 8, 8, 8, 8, 8, 8)
 (5, 5, 6, 8, 8, 8, 8, 9, 9)
 (5, 5, 7, 8, 8, 8, 8, 8, 9)
 (6, 6, 6, 7, 8, 8, 8, 8, 9)

(6, 6, 6, 8, 8, 8, 8, 8)
 (6, 6, 7, 7, 7, 7, 8, 9, 9)
 (6, 6, 7, 7, 7, 8, 8, 8, 9)
 (6, 6, 7, 7, 8, 8, 8, 8, 8)
 (6, 7, 7, 7, 7, 8, 8, 8, 8)
 (7, 7, 7, 7, 7, 7, 8, 9)
 (7, 7, 7, 7, 7, 8, 8, 8)

23 : (5, 5, 7, 7, 9, 9, 9, 9, 9)
 (5, 5, 7, 8, 8, 9, 9, 9, 9)
 (5, 5, 8, 8, 8, 8, 9, 9, 9)
 (5, 8, 8, 8, 8, 8, 8, 8, 8)
 (6, 6, 7, 8, 8, 8, 8, 9, 9)
 (6, 6, 8, 8, 8, 8, 8, 8, 9)
 (6, 7, 7, 7, 8, 8, 8, 9, 9)
 (6, 7, 7, 8, 8, 8, 8, 8, 9)
 (6, 7, 8, 8, 8, 8, 8, 8, 8)
 (7, 7, 7, 7, 7, 7, 9, 9, 9)
 (7, 7, 7, 7, 7, 8, 8, 9, 9)
 (7, 7, 7, 7, 8, 8, 8, 8, 9)
 (7, 7, 7, 8, 8, 8, 8, 8, 8)

24 : (6, 6, 7, 8, 9, 9, 9, 9, 9)
 (6, 6, 8, 8, 8, 9, 9, 9, 9)
 (6, 7, 7, 7, 9, 9, 9, 9, 9)
 (6, 7, 7, 8, 8, 9, 9, 9, 9)
 (6, 7, 8, 8, 8, 8, 9, 9, 9)
 (6, 8, 8, 8, 8, 8, 8, 9, 9)
 (7, 7, 7, 7, 8, 9, 9, 9, 9)
 (7, 7, 7, 8, 8, 8, 9, 9, 9)
 (7, 7, 8, 8, 8, 8, 8, 9, 9)
 (7, 8, 8, 8, 8, 8, 8, 8, 9)
 (8, 8, 8, 8, 8, 8, 8, 8, 8)

25 : (7, 7, 7, 9, 9, 9, 9, 9, 9)
 (7, 7, 8, 8, 9, 9, 9, 9, 9)
 (7, 8, 8, 8, 8, 9, 9, 9, 9)
 (8, 8, 8, 8, 8, 8, 9, 9, 9)

26 : (8, 8, 8, 9, 9, 9, 9, 9, 9)

27 : (9, 9, 9, 9, 9, 9, 9, 9, 9)

6 Triplet Puzzles

Given

→ a triplet grid:

A	B	C
D	E	F
G	H	I
J	G	K
K	J	G
G	K	J
L	M	N
N	L	M
M	N	L

→ and possibly further info:

- $G = \langle \dots \rangle$
- $\text{segment23} = (\dots)$
- L, M, N are equivalence classes mod 3
- given digit(s)
- constraint X
-

Normal Sudoku rules apply – gerðu svo vel!